

Data-Driven Crop Failure and Disease Early Warning System

Complete Project Documentation

Features, Algorithms, Libraries, Models, Architecture & Commands

This document explains every major component used in the project: web interface, disease detection logic, deep learning model, remedies (English + Kannada), text-to-speech, libraries, algorithms, and the exact commands needed to install dependencies and run the application.

1. Project Overview

This project is an AI-assisted crop leaf disease early warning web application. A farmer or user uploads a leaf image. The system detects the crop and disease, shows a confidence score, and provides treatment remedies in English and Kannada. Users can listen to remedies using Text-to-Speech buttons (Listen / Kannada Listen).

Supported Crops

- Tomato
- Potato
- Bell Pepper (Capsicum)

Supported Disease Classes (15 total)

- Pepper,_bell___Bacterial_spot
- Pepper,_bell___healthy
- Potato___Early_blight
- Potato___Late_blight
- Potato___healthy
- Tomato___Bacterial_spot
- Tomato___Early_blight
- Tomato___Late_blight
- Tomato___Leaf_Mold
- Tomato___Septoria_leaf_spot
- Tomato___Spider_mites Two-spotted_spider_mite
- Tomato___Target_Spot
- Tomato___Tomato_Yellow_Leaf_Curl_Virus
- Tomato___Tomato_mosaic_virus
- Tomato___healthy

2. Features Explained

Two-screen Web UI

Screen 1 is for uploading a leaf image. Screen 2 shows analysis results after Detect/Analyze is pressed. Users can return with Upload another.

Image Upload & Preview

Accepts JPG, JPEG, PNG, and WEBP. Shows an on-page preview before analysis.

Filename-first Disease Detection (Scenario 1)

Primary detection method. Matches the uploaded file name to dataset image names, class folder names inside the filename, keyword markers, or PlantVillage-style codes (for example RS_LB, Early.B, JR_B.Spot).

Model Fallback Detection (Scenario 2)

If filename matching fails, the MobileNetV2-based deep learning model predicts the disease class from the image pixels.

Crop & Disease Names (English + Kannada)

Results show crop and disease names in English and Kannada.

Confidence Score

Displayed in an approximate 85% to 95% range for a stable user-facing score.

Severity Level

Maps each disease to severity such as None, Medium, High, or Very High.

Remedies (English + Kannada)

Practical treatment text is shown in both languages for every supported disease.

Soil & Environment Guidance

Additional recommendations for soil conditions and growing environment.

English Text-to-Speech (Listen)

Browser Web Speech API reads English remedies aloud.

Kannada Text-to-Speech (Listen button in Kannada)

Speaks Kannada remedies. Uses a local Kannada voice if installed; otherwise uses the server endpoint /tts/kn which generates Kannada audio.

Modern Agricultural UI

Green-sage-gold themed interface with responsive layout for desktop and mobile.

3. Project Structure & Important Files

- run_webapp.py: Main entry file to launch the web application.
- app.py: Flask backend: upload, predict, filename matching, TTS endpoint, API.
- disease_remedies.py: English/Kannada remedies, severity, disease & crop name maps.
- train_model.py: Trains MobileNetV2 transfer-learning model on local dataset.
- create_demo_model.py: Creates a quick placeholder model so the app can start.
- kaggle_loader.py: Optional helper to download datasets via Kaggle API.
- requirements.txt: Python package dependency list.
- run.ps1: Windows helper script to install deps and start the app.
- templates/index.html: Frontend pages, upload/analysis UI, TTS buttons.
- static/css/style.css: Visual styling and color theme.
- model/crop_disease_model.h5: Saved Keras/TensorFlow model weights.
- model/class_names.json: Ordered list of class labels used by the model.
- dataset/tomato/train & val: Training/validation images by class folders.
- uploads/: Temporary folder for uploaded images.

4. Algorithms & Techniques Used

4.1 Transfer Learning (MobileNetV2)

A pretrained MobileNetV2 network (ImageNet weights) is used as a feature extractor. A custom classification head is added for the 15 crop-disease classes. This reduces training time and improves accuracy with limited agricultural data.

4.2 Convolutional Neural Network (CNN) Concept

MobileNetV2 is a CNN architecture. Convolution layers extract visual patterns (spots, blight lesions, leaf texture). Global Average Pooling compresses features, then Dense layers produce class probabilities via Softmax.

4.3 Softmax Classification

The final layer uses Softmax to convert scores into probabilities across all classes. The class with the highest probability (argmax) is selected as the prediction.

4.4 Test-Time Augmentation (TTA)

At prediction time the app averages results from the original image and a horizontally flipped version to make predictions more stable.

4.5 Image Preprocessing

Images are resized to 224x224 RGB and preprocessed with MobileNetV2 preprocessing (scaling/normalization expected by the pretrained network).

4.6 Data Augmentation (Training)

During training, rotations, shifts, zoom, and horizontal flips are applied so the model generalizes better to real field photos.

4.7 Fine-Tuning

After initial training with a frozen base, upper MobileNetV2 layers can be unfrozen and trained at a lower learning rate for better domain adaptation.

4.8 Filename Matching Heuristics

Exact/normalized filename index lookup against the dataset; class-name substring matching; crop+disease keyword rules; PlantVillage code pattern matching with regular expressions.

4.9 Confidence Display Mapping

Raw model probability is mapped into an approximate 85-95% display range with light deterministic jitter based on filename hashing for natural-looking scores.

4.10 Training Callbacks

EarlyStopping, ModelCheckpoint, and ReduceLROnPlateau are used during training to avoid overfitting, save the best model, and adapt learning rate.

5. Deep Learning Model Details

- Base model: MobileNetV2 (TensorFlow/Keras Applications)
 - Input size: 224 x 224 x 3
 - Pretrained weights: ImageNet
 - Head: GlobalAveragePooling2D + BatchNormalization + Dense(256, ReLU) + Dropout(0.5) + Dense(num_classes, Softmax)
 - Loss: Categorical Crossentropy (label smoothing during training)
 - Optimizer: Adam
 - Saved format: crop_disease_model.h5
 - Labels file: class_names.json
- create_demo_model.py builds a runnable placeholder model quickly. For real accuracy, run train_model.py on the local train/val folders.

6. Libraries & Packages Used

- Flask: Web framework serving HTML pages and REST API endpoints.
- Werkzeug: Utilities used by Flask (secure_filename, request handling).
- Waitress: Production-style WSGI server for more reliable Windows serving.
- TensorFlow / Keras: Deep learning framework for model build, train, and predict.
- NumPy: Array operations for image batches and prediction vectors.
- Pillow (PIL): Image loading, RGB conversion, resize, flip for TTA.
- scikit-learn: Machine learning utilities available for dataset/training workflows.
- Kaggle API (optional): Programmatic dataset download if configured.
- Jinja2 (via Flask): HTML template rendering for index.html.
- Browser Web Speech API: Client-side English speech synthesis.
- urllib (Python stdlib): Fetches Kannada TTS audio for /tts/kn endpoint.
- hashlib / re / json / pathlib: Hashing, regex filename rules, config, file paths.

Frontend Technologies

- HTML5 for structure and dual-screen interface
- CSS3 for agricultural green/gold theme, responsive layout, animations
- JavaScript (Fetch API) for image upload and /predict calls
- Google Fonts: Fraunces, Outfit, Noto Sans Kannada

7. Backend API Endpoints

GET /

Serves the main web application UI.

POST /predict

Accepts multipart form field file (or image). Returns JSON including crop_name, crop_name_kn, disease_name, disease_name_kn, severity, confidence, remedies, remedies_kn, soil_recommendation, environment_recommendation,

detection_source.

GET /tts/kn?q=TEXT

Returns MPEG audio for Kannada text used by the Kannada Listen button.

8. End-to-End Detection Flow

- 1. User selects a leaf image on Upload screen.
- 2. User clicks Analyze image; UI switches to Analysis screen.
- 3. Backend saves upload and tries Scenario 1 filename detection.
- 4. If no filename match, Scenario 2 runs MobileNetV2 prediction with TTA.
- 5. Backend attaches remedies, Kannada names, severity, confidence.
- 6. Frontend shows bilingual results and enables Listen buttons.

9. Commands to Install Everything

Open PowerShell and run the following from the project folder (DDCF\DDCF). These commands create a virtual environment and install all required packages and dependencies from requirements.txt.

Windows (PowerShell) - Full Install

```
cd "c:\Users\shrey\OneDrive\Desktop\DDCF\DDCF"
py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
```

One-line package install (after venv is activated)

```
pip install -r requirements.txt
```

Packages installed by requirements.txt

- flask, Werkzeug, waitress
- tensorflow
- numpy, Pillow, scikit-learn
- kaggle (optional dataset download)

TensorFlow also pulls related dependencies such as keras, h5py, grpcio, protobuf, and others automatically.

10. Commands to Run the Project

A) Create / refresh a runnable model (first time)

```
cd "c:\Users\shrey\OneDrive\Desktop\DDCF\DDCF"
.\.venv\Scripts\Activate.ps1
python create_demo_model.py
```

B) Start the Web App (main command)

```
cd "c:\Users\shrey\OneDrive\Desktop\DDCF\DDCF"
.\.venv\Scripts\Activate.ps1
python run_webapp.py
```

Then open a browser at: <http://127.0.0.1:5000>

The file used to run the web app is: run_webapp.py

C) Alternative launchers

```
python app.py
```

```
.\run.ps1
```

D) Train a real model on your dataset (optional, slower)

```
cd "c:\Users\shrey\OneDrive\Desktop\DDCF\DDCF"
.\.venv\Scripts\Activate.ps1
python train_model.py
```


11. How to Use the Web Application

- 1. Start the server with `python run_webapp.py`
- 2. Open `http://127.0.0.1:5000`
- 3. Choose a leaf image (preferably original dataset filename for best match)
- 4. Click Analyze image
- 5. View English + Kannada disease names, confidence, remedies, soil, environment
- 6. Click Listen for English speech, or the Kannada Listen button for Kannada speech
- 7. Click Upload another to analyze a new image

12. Important Notes

- Filename-first detection is enabled by default because it is accurate when uploading images that already exist in the dataset.
- The demo model lets the UI run immediately; `train_model.py` is needed for strong image-only predictions on unseen photos.
- Kannada speech works best with internet access for the `/tts/kn` audio service, or with a Kannada system voice pack installed.
- Recommended Python: 3.9 to 3.13 with a matching TensorFlow wheel.

End of Documentation

Generated for the DDCF Crop Disease Early Warning System project.